Multi-Objective Photoreal Simulation (MOPS) Dataset for Computer Vision in Robot Manipulation

Maximilian Xiling Li **Paul Mattes** **Nils Blank** **Rudolf Lioutikov**
Intuitive Robots Lab, KIT | Robotics Institute Germany
{m.li, lioutikov}@kit.edu

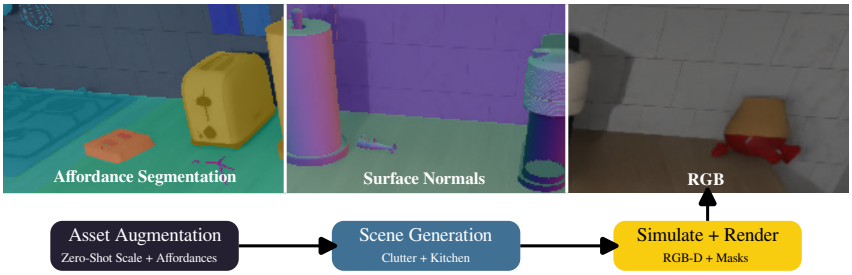


Figure 1: MOPS provides photorealistic scenes with pixel-perfect ground truth for robotic perception. Top: RGB, surface normals, and multi-label affordance segmentation from a single scene. Bottom: the three-stage generation pipeline using LLM-based zero-shot asset augmentation.

Abstract

Datasets providing high-quality visual annotations in manipulation-relevant scenes remain scarce. We introduce MOPS, a dataset generation framework that combines 3D assets from PartNet-Mobility and RoboCasa with a zero-shot LLM-based augmentation pipeline to automatically normalize object scale and generate part-level affordance annotations, describing how an object part can be manipulated (e.g., a mug handle is "graspable," a drawer is "pullable"). Built on ManiSkill3, MOPS produces photorealistic indoor scenes with pixel-perfect ground truth for class, part, and instance segmentation, multi-label affordances, depth, surface normals, and 6D poses, spanning 54 affordance types across 137 object categories. Human verification confirms 97.3% accuracy of the zero-shot affordance labels. We validate MOPS on three vision benchmarks of increasing scene complexity and show that ground-truth affordance masks improve imitation learning success rates on 24 RoboCasa manipulation tasks by 7.9 percentage points over RGB-only baselines, with predicted affordances still yielding measurable gains. The dataset and framework will be publicly available.

1 Introduction

Machine learning methods in computer vision rely on task-specific datasets for training and evaluation, spanning applications from affordance segmentation [1] and 3D part segmentation [2] to scene graph generation [3] and 6D pose estimation [4]. However, creating comprehensive datasets with detailed annotations requires substantial human effort, limiting both scale and annotation granularity. Moreover, the robotics domain remains underrepresented: embodied agents require robust perception of actionable object parts and manipulation opportunities, yet few vision datasets address these needs.

Datasets for learning computer vision for robot manipulation should ideally fulfill several key requirements:

Preprint.

1. **Manipulation-relevant Objects** (REQ OBJ): Common household items.
2. **Manipulation-relevant Annotations** (REQ ANN): High-resolution labels including part information, visual affordance labels, and 6D poses.
3. **Manipulation-relevant Representations** (REQ REP): Beyond images, incorporating scene representations such as pointclouds.
4. **Manipulation-relevant and Realistic Environments** (REQ ENV): Photorealistic rendering with natural clutter, beyond controlled laboratory conditions.
5. **Manipulation-relevant Interactions** (REQ INT): Agent-object and object-object interactions over time, supporting policy evaluation or active perception.

Existing datasets fulfill merely a subset of these requirements. Human-centric video datasets [5] capture dynamic interactions but are unsuitable for robot policy training. Affordance [1] and pose estimation [4] datasets lack environmental realism and scene dynamics. Robot datasets [6, 7] feature realistic scenes but provide insufficient ground truth for vision tasks, requiring post-hoc labeling [8]. To the best of our knowledge, no existing framework provides comprehensive pixel-wise ground truth for semantic concepts and affordances out of the box.

This work introduces **M**ulti-**O**bjective **P**hotoreal **S**imulation (MOPS), a dataset generation framework addressing all five requirements. MOPS combines assets from PartNet-Mobility [9] and RoboCasa [10] with a zero-shot augmentation pipeline built on GPT-4o [11] that automatically normalizes 3D object scale and generates part-level affordances (REQ OBJ, REQ ANN). Scenes are photorealistically rendered via ManiSkill3 [12] with pixel-perfect ground truth for class, part, and instance segmentation, affordance labels, surface normals, depth, and 6D poses (REQ ENV, REQ REP). Built on a physics simulator, MOPS supports interactive robot learning and teleoperated demonstration recording (REQ INT). Figure 1 illustrates example annotations and the generation pipeline.

We validate MOPS through vision benchmarks on three dataset variants of increasing complexity (MOPS-Object, MOPS-Clutter, MOPS-Kitchen) and through imitation learning experiments on 24 RoboCasa manipulation tasks. Our results show that ground-truth affordance masks improve manipulation success rates by 7.9 percentage points over RGB-only baselines, and that even predicted affordances yield measurable gains, demonstrating that MOPS provides supervision signals that directly transfer to robot behavior.

In summary, our contributions are:

- A zero-shot augmentation pipeline using LLMs for automatic scale normalization and affordance annotation of 3D assets, validated at 97.3% accuracy by human annotators
- A dataset generation framework producing unlimited photorealistic scenes with pixel-perfect ground truth for 54 affordances across 137 object categories
- Vision benchmarks across three dataset variants revealing that frozen self-supervised features (DINOv2) outperform task-specific fine-tuning in complex cluttered scenes
- Imitation learning experiments showing that MOPS-derived affordance annotations improve robot manipulation success rates over RGB-only policies

2 Related Work

Vision Datasets: The computer vision community has developed specialized datasets for various tasks, including fine-grained image classification with CUB-200-2011 [13], which provides RGB images of 200 bird species with point-based annotations for body parts and attributes. For semantic segmentation, datasets like Cityscapes [14] and SemanticKITTI [15] focus on autonomous driving scenarios. While these datasets excel for their specific tasks, their content is less relevant to robot manipulation (REQ OBJ).

Indoor datasets like ScanNet++ [16] and Hypersim [17] provide closer domain coverage with RGB-D scans and photorealistic renders, respectively, but offer only class and instance segmentations, lacking affordance annotations, 6D poses, and interactivity (REQ ANN, REQ INT). The RGB-D Part Affordance dataset [1] addresses affordance detection with single-object RGB-D images, but suffers from limited environmental realism with uniform backgrounds and only three cluttered scenes (REQ ENV).

Table 1: **Related Work Overview.** Comparison of computer vision and robotics datasets. MOPS bridges the gap by providing comprehensive part-level affordance annotations (S+I+P+A) across multiple modalities (R+D+M).

Dataset	Objects	Annotations	Represent.	Env.	Interact.	Trajectory
Vision Datasets						
CUB-200-2011 [13]	—	P*	R	—	—	—
CityScapes [14]	—	S+I	R	—	—	—
SemanticKITTI [15]	—	S+I	R+D+M	—	—	—
ScanNet++ [16]	✓	S+I	R+D+M	✓	—	—
HyperSim [17]	✓	S+I	R+D+M	✓	—	—
RGB-D Part Aff. [1]	✓	A	R+D	—	—	—
3D AffordanceNet [18]	✓	A	M	—	—	—
PartNet-Mobility [9]	✓	P	M	—	—	—
Robotics Datasets						
2-Handed [19]	✓	S+I	R+D	✓	—	—
Open-X [6]	✓	—	R+D	✓	—	✓
DROID [7]	✓	—	R+D	✓	—	✓
AI2-THOR [20]	✓	S+I	R+D+M	✓	✓	—
Behavior-1K [21]	✓	S+I	R+D+M	✓	✓	✓
RoboCasa [10]	✓	S+I	R+D+M	✓	✓	✓
Agibot-World [22]	✓	S+I	R+D+M	✓	✓	✓
InternA1 [23]	✓	S+I	R+D+M	✓	✓	✓
MolmoBot [24]	✓	S+I	R+D+M	✓	✓	✓
MOPS (Ours)	✓	**S+I+P+A**	**R+D+M**	✓	✓	(✓)

Abbreviations: **S**: Semantic Seg., **I**: Instance Seg., **P**: Part Seg., **P***: Part Centers, **A**: Affordance Seg., **R**: RGB, **D**: Depth, **M**: 3D Mesh or Pointcloud).(✓) denotes compatibility with external trajectories (RoboCasa).

3D Asset Datasets: 3D AffordanceNet [18] provides nearly 23k 3D point clouds across 23 object categories with 18 affordance labels (REQ OBJ, REQ ANN) but lacks material information crucial for photorealistic rendering (REQ ENV). PartNet-Mobility [9] addresses this gap by providing ShapeNet objects with material information and modeled articulation suitable for physics simulation (REQ INT), yet lacks affordance annotations (REQ ANN). A critical limitation shared by both libraries is inconsistent scaling—objects are not modeled to a common reference scale and often appear disproportionate relative to robotic manipulators. MOPS addresses these limitations through a zero-shot LLM pipeline that enriches PartNet-Mobility assets with affordance annotations and normalizes scale to realistic proportions.

Robotics Datasets: Real-world datasets such as Open X-Embodiment [6] and DROID [7] provide robot trajectories in realistic scenes, but scaling them to typical vision dataset sizes remains prohibitively expensive, and obtaining high-quality ground truth annotations requires manual labeling or post-hoc pipelines [8]. Simulation frameworks offer more scalable alternatives. AI2-THOR [20], Behavior-1K [21], and RoboCasa [10] provide complex room-scale scenes with photorealistic rendering (REQ ENV, REQ REP), while RoboCasa additionally offers over 100k training trajectories. Generative simulation frameworks [25, 26] leverage generative AI for near-unlimited scene variety, but perform best with simple task prompts rather than cluttered environments, limiting their effectiveness for vision applications (REQ ENV). Recent platforms such as AGIBOT-World [22], InternA1 [23], and MolmoBot [24] provide large-scale synthetic trajectories to bridge the sim-to-real gap. However, to the best of our knowledge, no simulation framework provides comprehensive pixel-wise ground truth annotations for semantic concepts and affordances out of the box.

MOPS combines RoboCasa's high scene variety [10] with zero-shot augmented assets from PartNet-Mobility [9], enabling generation of virtually unlimited realistic scenes with pixel-wise ground truth annotations, including affordances, in cluttered environments across multiple representations. Built on the ManiSkill3 simulator [12], MOPS improves visual quality and generation speed through raytracing and GPU parallelization. Table 1 provides a comparison of current datasets against manipulation-relevant requirements.

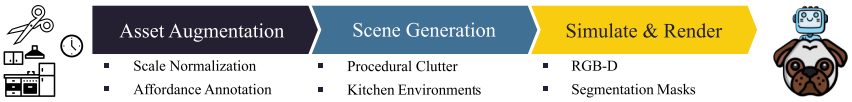


Figure 2: The three stages of the MOPS dataset creation process, which uses 3D assets from RoboCasa and PartNet-Mobility.

3 Methodology Overview

The MOPS pipeline (Figure 2) has three stages: **(1) Asset Augmentation**: a zero-shot LLM pipeline normalizes object scale and generates affordances (Section 4); **(2) Scene Generation**: augmented PartNet-Mobility assets are procedurally placed into RoboCasa kitchens to form realistic cluttered scenes (Section 5); **(3) Rendering and Annotation**: ManiSkill3's photorealistic renderer produces multi-modal observations with pixel-perfect ground truth for segmentations, 6D poses and multilabel affordances.

4 Zero-Shot 3D Asset Augmentation

MOPS creates photorealistically rendered scenes of indoor environments for robot manipulation (REQ ENV) by using the ManiSkill3 [12] simulator and renderer. To populate the scenes with realistic and interactive objects (REQ OBJ) MOPS uses 3D assets provided by RoboCasa [10] and PartNet-Mobility [9]. However, the 3D assets must first be augmented for use in MOPS. Firstly, 3D objects are not modeled on the same reference scale, breaking realism (REQ OBJ & REQ ENV). Secondly, the 3D assets must be annotated with affordance annotations in order to easily provide pixel-wise ground truth masks (REQ ANN). In order to address these issues, MOPS employs a zero-shot augmentation pipeline based on GPT-4o [11].

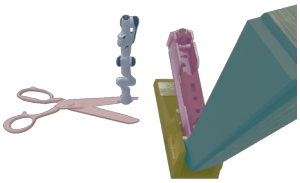


Figure 3: Unscaled 3D assets exhibit unrealistic sizes relative to the robot. Colored overlays indicate generated part-level affordances.

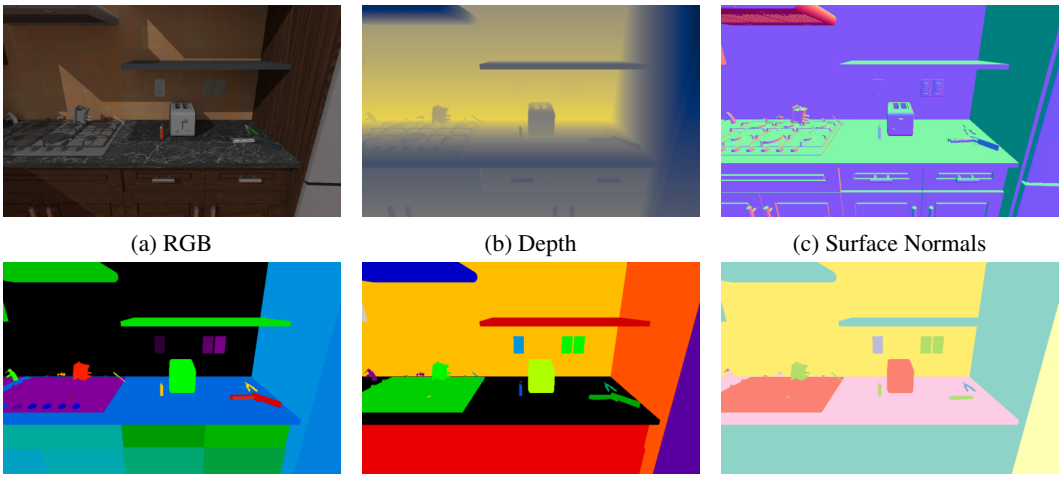
4.1 Zero-Shot Asset Normalization (REQ OBJ)

MOPS uses 3D assets of common household objects. However, most 3D assets are not modeled on the same reference scale. This results in unrealistic relative sizes between different assets or in relation to a simulated robot. We propose a zero-shot asset normalization stage to address this issue. We first verify the internal, spatial scale of the simulation by comparing the Denavit-Hartenberg parameters of a Franka Emika Panda 7-DoF arm [27] with the endeffector coordinates in simulation. This confirms a simulation scale of 1.0 simulation units equaling 1.0 meters.

Each asset in PartNet-Mobility is annotated with XYZ-bounding boxes and an object category. Through the common-sense knowledge embedded in recent LLM, MOPS can obtain realistic object scales. In particular, GPT-4o is queried to output standard dimensions for each object category. MOPS queries the LLM for realistic minimum and maximum Width $\times$ Height $\times$ Depth (WHD) sizes for each object category. As the orientation of each asset is unknown, the calculation process divides the largest bounding box dimension with the largest WHD dimension to obtain realistic scaling factors $s_{\min}$ and $s_{\max}$. When loading an asset instance into the simulation, we sample a random scaling factor from the range $[s_{\min}, s_{\max}]$ to increase visual variety within a scene. Figure 3 presents an example scene before scale normalization. This approach enables easy addition of new object categories and models into the dataset without human intervention.

4.2 Zero-Shot Affordance Annotation (REQ ANN)

MOPS integrates the PartNet-Mobility dataset, a collection of articulated, part-based 3D assets. It is a subset of the PartNet asset dataset [28], which in turn is a subset of ShapeNet [2]. The fully articulated PartNet-Mobility assets provide a wide range of interactions (REQ INT), but are missing manipulation annotations such as affordances (REQ ANN).



(a) RGB (b) Depth (c) Surface Normals

(d) Part Segmentation (e) Class Segmentation (f) Multilabel Affordances

Figure 4: MOPS provides multiple, pixelwise ground truth maps in addition to RGB-D perception.

MOPS again leverages the common-sense reasoning capabilities of LLMs to generate multi-label affordances for each object on a part and object level. Particularly, GPT-4o is queried to output a list of affordances for each object and object part. For 3D assets without part annotations, e.g., the RoboCasa assets, the zero-shot annotation stage labels the entire object with all applicable affordances.

Finally, MOPS clusters the affordances with a sentence embedding model to filter duplicates such as *closable* and *close* and aligns semantic clusters like *heatable* and *warmup-able*. Future work could extend the annotation stage to produce region-level affordance annotations similar to the manually labeled 3D AffordanceNet [18]. Meanwhile, the already presented zero-shot annotation stage alleviates the human-labeling effort significantly. The exact prompts for the zero shot 3D asset augmentation pipeline are detailed in Appendix B.

5 MOPS Dataset Generation

MOPS can generate an unlimited number of scenes by combining 120 realistic indoor environments from RoboCasa and its assets with 2,300 articulated objects provided by PartNet-Mobility and augmentations from a zero-shot asset augmentation pipeline.

Throughout this work, *interaction* refers to physics-enabled object manipulation (e.g., opening drawers, articulating joints), while *robot trajectory* refers to sequences of robot actions, such as joint configurations, end-effector poses and gripper commands during task execution.

5.1 MOPS Ground Truth Masks (REQ ANN)

MOPS provides multiple scene representations and camera views to facilitate training for robot manipulation tasks. Camera configurations include birds-eye view (for SLAM reconstruction in mobile manipulation), and external over-the-shoulder, ego, and in-hand views (mimicking typical robot manipulation setups).

MOPS cameras provide commonly used image modalities. The RGB renderings of the simulated scene can use raytracing for enhanced realism, or employ rasterized rendering for faster computation. The simulation provides depth images with distance to the camera in millimeters, providing 2.5D RGB-D inputs or ground truth data for learning depth estimation. Additionally, the underlying simulation provides ground truth surface normal maps and part level segmentation masks for the simulated assets.

MOPS can provide additional ground truth modalities from the rendering, such as 6D poses for objects in the camera frame, or instance and semantic segmentation masks. These masks are generated via look-up tables MOPS populates during scene creation and provides pixel-perfect ground truth

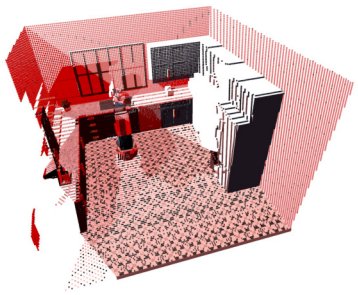


Figure 5: Pointcloud of a RoboCasa scene with red ambient lighting.

Figure 6: A RoboCasa kitchen filled with PartNet-Mobility clutter.

data. By obtaining affordance labels from its zero-shot asset augmentation pipeline, MOPS also provides pixel-perfect affordance annotations for each camera.

Figure 4 presents different, visual camera modalities from an ego camera in a sparsely populated RoboCasa scene. The RGB modality is rendered via raytracing, resulting in realistic shadows and reflections. The depth modality shows the distance of each pixel to the virtual camera. The surface normal modality encodes the XYZ components of each surface normal in the RGB values of the image. For the part segmentation, each pixel stores the simulation internal part ID, whereas the class segmentation performs a table lookup to retrieve the class for each part ID. Similarly, for instance segmentation, a different lookup-table storing object instances is used. The difference between these maps is most notable for articulated objects, like the cabinet door or robot arm, where each articulation link is a separate part, but share the same class. Finally, affordances are modeled in MOPS as a multi-label annotation; consequently, the annotation map output is a per-pixel binary vector indicating the presence or absence of each affordance type.

5.1.1 Extended Representations (REQ REP)

The underlying ManiSkill3 simulation supports point cloud generation by merging the 2.5D images of all cameras. The observations can be further customized by changing the extrinsic and intrinsic parameters of the virtual cameras or the lighting configuration. Figure 5 shows a scene point-cloud with red ambient lighting.

The SAPIEN renderer provides realistic stereo depth sensors with active IR lighting and simulated sensor noise [29]. While these sensors are not yet fully available in the current ManiSkill3 release[1], their integration into MOPS will be straightforward once released, based on prior experience with ManiSkill2.

5.2 MOPS Realistic Environments (REQ ENV)

The goal of MOPS is to generate a vision dataset of realistic environments relevant to robot manipulation. The high visual quality of the used RoboCasa assets aids in generating realistic indoor kitchen scenes. Additionally, the high-quality raytracing implementation used by ManiSkill3 results in photorealistic lighting and rendering.

In order to create realistically cluttered scenes with potential object overlap and distractors, MOPS procedurally places augmented PartNet-Mobility assets in the RoboCasa kitchen scenes. First, MOPS obtains the location of the kitchen counter tops from the simulation. Then, it computes the available space by observing the bounding box of the collision mesh. Finally, it generates random positions within the available space and drops simulation objects (see Figure 6). While this is a rather heuristic approach compared to trained models such as ClutterGen [30], it can be easily applied to novel scenes and environments outside of RoboCasa, without requiring any training. More scenes are presented in Appendix G.

5.3 Interactive Simulation (REQ INT)

[1] ManiSkill v3.0.0b22, retrieved 2026-04-22 from GitHub

MOPS leverages ManiSkill3 for full interactivity. While MOPS primarily uses existing robot trajectories from RoboCasa (see Table 1), it also supports recording new demonstrations through ManiSkill3's teleoperation interface.

RoboCasa demonstrations can be rolled out to record observations in dynamic environments with full physics simulation. Figure 7 illustrates a teleoperated demonstration where the robot depresses the hinged top of a stapler. Recorded trajectories can be replayed to generate the comprehensive ground truth segmentations provided by MOPS.

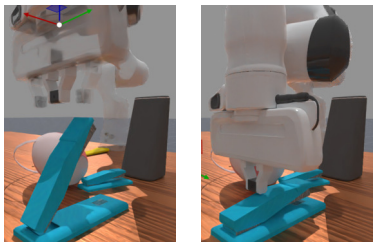


Figure 7: MOPS supports the mouse-and-keyboard teleoperation interface by ManiSkill3.

Table 2: Comparison of dataset diversity. While 3D-AffordanceNet has more instances, MOPS provides significantly higher taxonomic coverage across object categories and affordance types.

Dataset	Level	Aff. Labels	Obj. Cat.	Objects
RGB-D Part	Part	7	17	105
3D-AffNet	Part	16	23	**22,949**
MOPS-Partnet	Part	22	46	2,345
MOPS-Robocasa	Object	44	101	1,008
MOPS (Total)	**Mixed**	**54**	**137**	3,353

6 Affordance Analysis

MOPS provides comprehensive affordance annotations across multiple granularities. We derive part-level annotations from PartNet-Mobility [9], contributing 22 affordance types for 14,100 parts across 2,345 objects. To extend the semantic reach of our benchmark, we further incorporate object-level annotations for RoboCasa assets with 44 affordance types across 101 categories. By combining the both asset sets, **MOPS offers 54 affordance labels for 137 different object categories**.

To validate the zero-shot labeling pipeline, two human annotators manually verified all 14,100 generated part annotations. The annotators found the zero-shot pipeline to be highly accurate, confirming 97.3% of the labels. The 374 annotations (2.7%) found to be mislabeled, such as a fixed door frame erroneously assigned push or pull affordances, were subsequently corrected to ensure the fidelity of the benchmark.

While existing benchmarks like 3D-AffordanceNet [18] provide a higher volume of individual object instances, MOPS prioritizes taxonomic diversity. As shown in Table 2, MOPS offers $3.5\times$ more affordance types and over $6\times$ the number of object categories compared to prior work. This makes MOPS the most diverse dataset available for affordance detection, shifting the focus from instance-level repetition to the long-tail distribution of real-world object interactions.

7 Experiments

We construct three benchmark datasets with systematic differences in scene composition and visual complexity. We use a randomized train-test splits per asset class, ensuring no 3D model overlap between splits to test generalization capabilities of the models. Each image uses randomized lighting (intensity, color, direction), object poses, and camera positions. To ensure physical plausibility, images are captured only after a physics settling period. Our generation pipeline achieves a throughput of ≈ 3500 images/hour on consumer-grade hardware. Qualitative examples for all settings are provided in Appendix G.

MOPS-Object consists of isolated assets centered on uniform backgrounds. It contains 30 images per category (15 train/15 test) across 46 balanced part-level classes, mirroring the structure of the RGB-D Part Affordance Dataset [1]. It uses 512×512 as resolution.

Table 3: **Object Classification Baselines on MOPS.** All CLIP results use the ViT-B/32 architecture.

Model	Training	Top-1 Acc. (%)
ResNet-50	Full	61.3
ViT-B/16	Full	63.5
CLIP	Frozen	40.7
CLIP	Full	**72.2**

Table 4: **Multi-Label Segmentation Baselines.** Models are evaluated on 54 affordance labels using macro-averaged mIoU and F1 scores.

		Clutter-5K		Kitchen-5K		Kitchen-100K	
Model	**Training**	**mIoU**	**F1**	**mIoU**	**F1**	**mIoU**	**F1**
DeepLabV3	Full	14.5	15.5	40.0	42.6	40.8	43.2
Segformer	Full	**15.0**	**16.2**	41.9	44.4	**41.3**	**43.7**
DINOv3	Frozen	13.5	14.3	**34.9**	**38.9**	36.4	39.9

MOPS-Clutter-5K evaluates affordance recognition in unstructured tabletop scenes. It features randomized arrangements of multiple assets (4000 training, 1000 test images). Images are captured in 640×480, mimicking common robot camera configurations.

MOPS-Kitchen-5k represents our most challenging setting, situating PartNet-Mobility assets within complex RoboCasa kitchen environments alongside various distractor objects (4000 training, 1000 test images). Images are captured in 640×480, mimicking common robot camera configurations.

MOPS-Kitchen-100k tests large-scale training capabilities. It is in makeup identical to MOPS-Kitchen-5k, but with 90,000 training samples and 10,000 test samples.

7.1 Vision Benchmark

We validate MOPS's utility through two complementary tasks: single-object classification on MOPS-Object (Table 3) and multi-label affordance segmentation on MOPS-Clutter and MOPS-Kitchen (Table 4). Comprehensive training hyperparameters are detailed in Appendix D.

Classification. We benchmark standard vision backbones, including ResNet-50 [31] and ViT-B/16 [32], against CLIP [33] (ViT-B/32) to evaluate the difficulty of the MOPS-Object dataset. We evaluate these models in two states: frozen and fully trained. While CLIP in a frozen state provides a baseline for general knowledge, we find that even the best-performing model requires full training on our dataset to achieve high accuracy, yielding a $+31.5\%$ absolute gain over the frozen baseline. This significant performance gap highlights the domain-specific challenges inherent in our benchmark.

Affordance Segmentation. To evaluate the complexity of the 54-class affordance task, we benchmark DeepLabV3 [34], Segformer [35], and DINOv3 [36]. We report mIoU macro-averaged across all classes, comparing fully trained models against a frozen backbone. The results demonstrate a clear distribution shift: While fully trained models like Segformer achieve strong performance in the controlled Clutter setting (mIoU: 16.2%), their performance degrades in the more complex Kitchen environment (mIoU: 22.1%). Interestingly, frozen DINOv3 shows superior robustness in the Kitchen setting (mIoU: 34.9% vs 22.1%), suggesting that self-supervised pretraining captures more generalizable features for complex, cluttered scenes than task-specific fine-tuning on simpler environments.

7.2 Robot Manipulation

To evaluate the utility of the MOPS-derived annotations for downstream robotics tasks, we conduct imitation learning experiments on 24 single-stage RoboCasa tasks [10] spanning pick-and-place, articulated object manipulation and multi-object rearrangement scenarios.

Table 5: **Robot Manipulation Performance.** Imitation learning results on 24 RoboCasa tasks, evaluated over 10 environment seeds each.

Policy Inputs	Success Rate	95% CI
RGB only (Baseline)	13.33%	[9.6%, 18.2%]
RGB + MOPS Oracle	**21.25%**	[16.5%, 21.4%]
RGB + Aff. Pred.	16.25%	[12.1%, 21.4%]

Experimental Setup. We train DiTFlow policies [37], a Diffusion Transformer policy [38] using Flow Matching, within the LeRobot framework [39]. Each of the 24 tasks uses 50 human teleoperated demonstrations for training.

We compare three conditions: (1) an RGB-only baseline DiTFlow policy conditioned on observations from three cameras (in-hand, left-shoulder, right-shoulder), (2) an oracle policy that additionally receives ground-truth MOPS affordance segmentation masks for each camera view, and (3) an affordance-predicting policy which receives only RGB inputs but is jointly trained to predict segmentation masks together with robot actions. Each task is evaluated across 10 randomized environment seeds with different object poses and clutter configurations. Training hyperparameters are detailed in Appendix E

Results. Table 5 presents the average success rates across all 24 tasks. The oracle policy, which receives ground-truth MOPS affordance masks at test time, achieves 21.25% success rate compared to 13.33% for the RGB-only baseline — a +7.92 percentage point absolute improvement. This establishes an upper bound on the benefit of affordance information and validates that MOPS provides supervision signal for a capability that directly transfers to robot behavior. The affordance-predicting policy, which must infer masks from RGB at test time, reaches 16.25%, a +2.92 percentage point improvement over the baseline, demonstrating that the affordance signal remains useful even when predicted rather than given. The gap between the oracle and the predicting policy highlights room for improving the affordance prediction module itself as a direction for future work.

8 Conclusion

We introduce MOPS, a dataset generation pipeline for learning computer vision for robotics. Combining PartNet-Mobility and RoboCasa assets with a GPT-4o-driven zero-shot augmentation pipeline, MOPS produces photorealistic scenes with pixel-level ground truth for class, part and instance segmentation, affordance labels and 6D poses, and yields 54 affordance labels across 137 object categories. Built on ManiSkill3, it supports full interactivity for recording, learning and evaluating robot behavior. We validate MOPS with vision benchmarks and show that ground-truth affordances improve RoboCasa manipulation success rates by 7 pp over RGB-only baselines.

Limitations and Future Work. MOPS inherits constraints from ManiSkill3. Notably, raytraced rendering cannot currently be combined with GPU parallelization, limiting throughput; future work could port MOPS to Isaac Lab [40] or Genesis [26]. The augmentation pipeline currently requires assets with existing materials; extending it to generate materials and textures would unlock the full PartNet [28] and ShapeNet [2] libraries. Finally, while our current robot experiments represent successful first implementations, fully realizing the potential of these affordances via deeper architectural integration remains an important direction for future work.

Broader Impact. MOPS accelerates robot vision research by reducing the financial and environmental costs of real-world data collection. However, over-reliance on simulation poses a key risk: policies optimized in MOPS may behave unpredictably when encountering unmodeled real-world dynamics. Closing sim-to-real gaps is therefore a safety prerequisite, not merely a technical one. Policies trained via MOPS should be treated as research artifacts requiring extensive physical validation before deployment in human-centric or industrial settings.

References

[1] Austin Myers, Ching L. Teo, Cornelia Fermüller, and Yiannis Aloimonos. Affordance detection of tool parts from geometric features. In *ICRA*, 2015.

[2] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.

[3] Rowan Zellers, Mark Yatskar, Sam Thomson, and Yejin Choi. Neural motifs: Scene graph parsing with global context. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5831–5840, 2018.

[4] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199*, 2017.

[5] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild, 2012. URL `https://arxiv.org/abs/1212.0402`.

[6] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.

[7] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset, 2024.

[8] Nils Blank, Moritz Reuss, Marcel Rühle, Ömer Erdinç Yağmurlu, Fabian Wenzel, Oier Mees, and Rudolf Lioutikov. Scaling robot policy learning via zero-shot labeling with foundation models. In *8th Annual Conference on Robot Learning*, 2024. URL `https://openreview.net/forum?id=EdVNB2kHv1`.

[9] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[10] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems*, 2024.

[11] OpenAI. Gpt-4o system card, 2024. URL `https://arxiv.org/abs/2410.21276`.

[12] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.

[13] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. Caltech-ucsd birds-200-2011. Technical Report CNS-TR-2011-001, 2011.

[14] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[15] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *Proc. of the IEEE/CVF International Conf. on Computer Vision (ICCV)*, 2019.

[16] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the International Conference on Computer Vision (ICCV)*, 2023.

[17] Mike Roberts, Jason Ramapuram, Anurag Ranjan, Atulit Kumar, Miguel Angel Bautista, Nathan Paczan, Russ Webb, and Joshua M. Susskind. Hypersim: A photorealistic synthetic dataset for holistic indoor scene understanding. In *International Conference on Computer Vision (ICCV) 2021*, 2021.

[18] Shengheng Deng, Xun Xu, Chaozheng Wu, Ke Chen, and Kui Jia. 3d affordancenet: A benchmark for visual object affordance understanding. In *proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1778–1787, 2021.

[19] Marvin Heidinger, Snehal Jauhri, Vignesh Prasad, and Georgia Chalvatzaki. 2handedafforder: Learning precise actionable bimanual affordances from human videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 14743–14753, October 2025.

[20] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. Ai2-thor: An interactive 3d environment for visual ai. *arXiv preprint arXiv:1712.05474*, 2017.

[21] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR, 2023.

[22] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Xindong He, Xu Huang, et al. Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025.

[23] Yang Tian, Yuyin Yang, Yiman Xie, Zetao Cai, Xu Shi, Ning Gao, Hangxu Liu, Xuekun Jiang, Zherui Qiu, Feng Yuan, Yaping Li, Ping Wang, Junhao Cai, Jia Zeng, Hao Dong, and Jiangmiao Pang. Interndata-a1: Pioneering high-fidelity synthetic data for pre-training generalist policy, 2025. URL `https://arxiv.org/abs/2511.16651`.

[24] Abhay Deshpande, Maya Guru, Rose Hendrix, Snehal Jauhri, Ainaz Eftekhar, Rohun Tripathi, Max Argus, Jordi Salvador, Haoquan Fang, Matthew Wallingford, Wilbert Pumacay, Yejin Kim, Quinn Pfeifer, Ying-Chun Lee, Piper Wolters, Omar Rayyan, Mingtong Zhang, Jiafei Duan, Karen Farley, Winson Han, Eli Vanderbilt, Dieter Fox, Ali Farhadi, Georgia Chalvatzaki, Dhruv Shah, and Ranjay Krishna. Molmob0t: Large-scale simulation enables zero-shot manipulation, 2026. URL `https://arxiv.org/abs/2603.16861`.

[25] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation, 2023.

[26] Genesis. Genesis: A universal and generative physics engine for robotics and beyond, December 2024. URL `https://github.com/Genesis-Embodied-AI/Genesis`.

[27] Franka Robotics. Franka Denavit–Hartenberg parameters. URL `https://frankaemika.github.io/docs/control_parameters.html#denavithartenberg-parameters`. Franka FCI Documentation.

[28] Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. PartNet: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[29] Sapien Developers. Sapien 3.0 documentation, 2024. URL `https://sapien-sim.github.io/docs/user_guide/rendering/depth_sensor.html`. Sapien StereoDepthSensor Documentation.

[30] Yinsen Jia and Boyuan Chen. Cluttergen: A cluttered scene generator for robot learning. In *8th Annual Conference on Robot Learning*, 2024.

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

[32] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL https://arxiv.org/abs/2010.11929.

[33] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.

[34] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation, 2017. URL https://arxiv.org/abs/1706.05587.

[35] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. *Advances in neural information processing systems*, 34:12077–12090, 2021.

[36] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DINOv3, 2025. URL https://arxiv.org/abs/2508.10104.

[37] Nur Muhammad Shafiullah and Daniel San José Pro. DiTFlow policy in LeRobot. GitHub Pull Request #680, https://github.com/huggingface/lerobot/pull/680, 2024. Accessed: 2026-01-26.

[38] Sudeep Dasari, Oier Mees, Sebastian Zhao, Mohan Kumar Srirama, and Sergey Levine. The ingredients for robotic diffusion transformers. *arXiv preprint arXiv:2410.10088*, 2024.

[39] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. https://github.com/huggingface/lerobot, 2024.

[40] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. doi: 10.1109/LRA.2023.3270034.

A Licenses

This work makes use of the following existing assets and thier respective licenses:

1. PartNet-Mobility, SAPIEN Partnet-Mobility Terms of Use `https://sapien.ucsd.edu/about`
2. ManiSkill 3.0.0b22, Apache 2.0 `https://github.com/haosulab/maniskill`
3. RoboCasa v0.2, MIT License `https://github.com/robocasa/robocasa`

B GPT-4o Labeling Prompts

For reproducibility, we provide the exact prompts used for GPT-4o annotation.

B.1 Scale Annotation

System prompt:

> I will provide you with the name of the object. Output the typical lengths, widths, and heights of the object. Output json with the following format:
>
> ```
> {"length": [low,high],
> "width": [low,high],
> "height": [low,high]}
> ```

B.2 Affordance Annotation

System prompt:

> You will be provided with an object and parts of the object. Assign an affordance to each object part. Examples for affordances are the following: grasp, contain, lift, openable, layable, sittable, support, pourable, move, display, pushable, pull, listen, wear, press, cut, stab. You are not restricted to these affordances. Output valid json

Few-shot example:

User: Object: Remote
Parts: button, remote_base, knob, rotation_button

Assistant:

```
{
  "button": [
    "press"
  ],
  "remote_base": [
    "grasp", "moveable", "layable"
  ],
  "knob": [
    "press", "rotate"
  ],
  "rotation_button": [
    "rotate"
  ]
}
```

B.3 Detailed Affordances

Table 6: **Part-Level Affordance Statistics (PartNet-Mobility).** Distribution of affordance types across the 46 part-level categories in the MOPS dataset.

Affordance	Count	Affordance	Count	Affordance	Count	Affordance	Count
close	106	grasp	1897	pourable	171	sittable	128
connect	11	insert	51	press	6658	stab	105
contain	1650	layable	1381	pull	1611	support	1907
cut	208	lift	245	pushable	1602	wear	195
display	189	move	1996	remove	51		
foldable	32	openable	2079	rotate	701		

Table 7: **Object-Level Affordance Statistics (RoboCasa).** Distribution of affordance types across the 101 object-level categories in the MOPS dataset.

Affordance	Count	Affordance	Count	Affordance	Count	Affordance	Count	Affordance	Count
adjustable	15	contain	99	grasp	994	mixable	64	recyclable	98
breakable	68	cookable	213	hold	565	mountable	16	roastable	98
close	33	coolable	56	juicy	122	move	376	rotate	37
connect	2	cuttable	575	lift	177	openable	150	scoopable	25
consumable	110	decorative	44	lockable	4	organize	47	sealable	116
display	44	drinkable	157	mashable	61	peelable	181	shareable	73
edible	483	fillable	188	placeable	75	pointable	39	spreadable	88
press	10	pourable	252	stackable	225	store	105	squeezable	127
support	85	throwable	100	toastable	50	washable	220		

C Compute Resources

All Experiments and Data Generation were performed on a single Desktop Workstation with Intel i7-12700 20-Core CPU, NVidia RTX 3070 (8GB) GPU, 32 GB RAM, Python 3.14 and Pytorch 2.9.1

D Vision: Training Details

For reproducibility, we provide detailed training configurations for all baseline models evaluated on the MOPS datasets.

Online Data augmentation: Random horizontal flip, random rotation (±10°), color jitter.

D.1 Image Classification

Table 8: Training hyperparameters for image classification models on MOPS-Object.

Parameter	**ResNet50**	**ViT-B/16**	**CLIP-FT**
Batch size	64	32	32
Optimizer	AdamW	AdamW	AdamW
Learning rate	1e-4	1e-4	1e-5
LR schedule	None	Cosine w/ warmup	None
Warmup epochs	-	12	-
Weight decay	0.01	0.05	0.05
Epochs	40	120	10
Augmentation	Yes	Yes	Yes
Loss function	Cross-entropy	Cross-entropy	Cross-entropy
Pretrained weights	ImageNet-1K	ImageNet-1K	OpenAI CLIP

D.2 Affordance Segmentation

Table 9: Training hyperparameters for segmentation models on MOPS-Clutter and MOPS-Kitchen.

Parameter	**DeepLabV3**	**SegFormer-B2**	**DinoV2**
Backbone	ResNet50	Transformer	ViT-B/14
Batch size	16	16	16
Optimizer	AdamW	AdamW	AdamW
Learning rate	1e-4	1e-4	1e-3
LR schedule	OneCycle	OneCycle	None
Weight decay	0.01	0.01	0.01
Epochs	200	200	20
Training mode	Full	Full	Linear probing
Loss function	Binary Cross Entropy		
Pretrained weights	COCO	ImageNet-1K	DINOv2

D.3 Evaluation

We evaluate our models using the following metrics and data preprocessing steps:

Classification Top-1 accuracy on a balanced 46-class test set.

Segmentation Mean IoU (mIoU) across 56 affordance labels.

Data Preprocessing All images are resized to 512×512. For classification, ImageNet normalization is applied, while segmentation models normalize images to $[0, 1]$.

E Imitation Learning Details

Our imitation learning experiments use DiTFlow policies [37] implemented in the LeRobot framework [39]. All experiments were performed on a single Desktop Workstation with AMD Ryzen 9 9950X CPU, Nvidia RTX 5090 (32GB) GPU, 128 GB RAM, Python 3.14 and Pytorch 2.9.1.

Table 10: Training hyperparameters for DiTFlow on robot learning tasks.

Parameter	**Value**
Backbone	ResNet34
Batch size	32
Optimizer	Adam
Learning rate	1e-4
LR schedule	Cosine with warmup
Loss function	Flow Matching

The policies receive the RGB observations from the inhand, left-shoulder and right-shoulder cameras with resolutions 256×256. The affordance aware policy additionally receives ground truth affordance segmentation masks for each camera sensor. Each model is trained for 50 epochs using 50 human teleoperated demonstrations for each of Robocasa's 24 single stage tasks.

F MOPS Dataset Structure

Our datasets are stored as sharded *Apache Parquet* files, written via the Hugging Face `datasets` library to guarantee full compatibility with the Hugging Face Hub, its Data Studio viewer, and streaming via `load_dataset()`. Each dataset is organized as a directory of Parquet shards split into `train/` and `test/` subdirectories. Every row in a shard corresponds to a single observation and contains all associated modalities as columns.

F.1 Directory Layout

The dataset follows the standard Hugging Face Parquet convention, where subdirectory names determine the split:

- `dataset_dir/`
  - `train/`
    - `00000.parquet`
    - `00001.parquet`
    - `...`
  - `test/`
    - `00000.parquet`
    - `...`
  - `dataset_info.json`

Each Parquet shard contains a fixed number of rows (default 500). The dataset can be loaded directly with:

```
from datasets import load_dataset
ds = load_dataset("parquet", data_dir="dataset_dir")
```

F.2 Column Schema

Every row in a Parquet shard represents a single observation, identified by a unique `image_id` (e.g., `image_000000`). The following table details the per-row column schema.

Table 11: Column schema of the Parquet dataset. Image columns use the `datasets.Image` feature and are automatically decoded to PIL images on access. Array columns store compressed `.npz` bytes and are decoded with `np.load(io.BytesIO(v))["data"]`.

Category	Column	Description
Identity	`image_id`	Unique string identifier (e.g., `image_000042`).
	`asset_id`	PartNet-Mobility asset ID of the primary object.
Images	`image`	RGB image stored as PNG bytes (`datasets.Image` feature).
	`semantic`	Pixel-level semantic class mask, lossless grayscale PNG (`datasets.Image`).
	`instance`	Pixel-level instance segmentation mask, grayscale PNG (`datasets.Image`).
	`part`	Pixel-level part segmentation mask, grayscale PNG (`datasets.Image`).
	`is_partnet`	Binary mask indicating PartNet-Mobility pixels, grayscale PNG (`datasets.Image`).
Arrays	`affordance`	Affordance segmentation mask, shape $(H, W, 56)$ multi-hot int. Stored as compressed `.npz` bytes (binary column).
	`depth`	Depth map, shape $(H, W, 1)$ float32. Stored as compressed `.npz` bytes (binary column).
	`normal`	Surface normal map, shape $(H, W, 3)$ float32. Stored as compressed `.npz` bytes (binary column).
Annotations	`bbox`	Bounding boxes as a JSON string of `[x, y, w, h, class_id]` lists.
Metadata	`render_params`	JSON string encoding rendering parameters (camera pose, lighting, split).
Labels	`class_name`	Object class name string (single-object datasets only).
	`class_idx`	Integer class index (single-object datasets only).

F.3 Encoding Details

Image columns (`image`, `semantic`, `instance`, `part`, `is_partnet`) use the `datasets.Image` feature type. They are stored on disk as PNG bytes within the Parquet file and are automatically decoded to PIL images by `load_dataset()`. Integer masks with values ≤ 255 use 8-bit grayscale; masks exceeding this range use 16-bit grayscale. Both are lossless.

Array columns (`affordance`, `depth`, `normal`) are stored as compressed NumPy `.npz` bytes in binary Parquet columns. This preserves the original shape and dtype losslessly while achieving strong compression – in particular, the affordance masks are extremely sparse (>99% zeros) and compress by approximately 100$\times$ under NumPy's deflate. Decoding requires only NumPy:

```
import numpy as np, io
arr = np.load(io.BytesIO(row["depth"]))["data"]
```

G MOPS Dataset Example Images

The following pages showcase some random training samples from the three generated Datasets MOPS-Object (Figure 8), MOPS-Clutter (Figure 9 and MOPS-Kitchen (Figure 10).

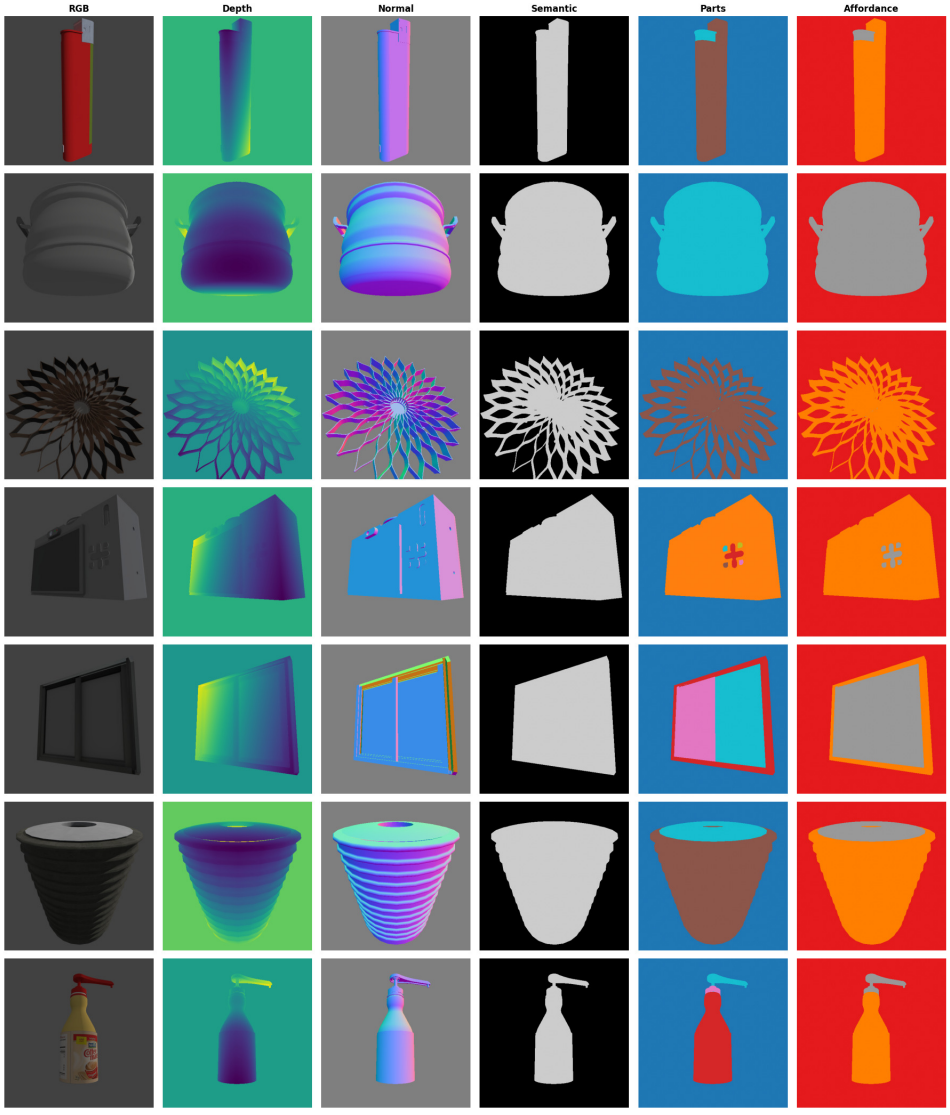


Figure 8: MOPS-Object dataset examples.

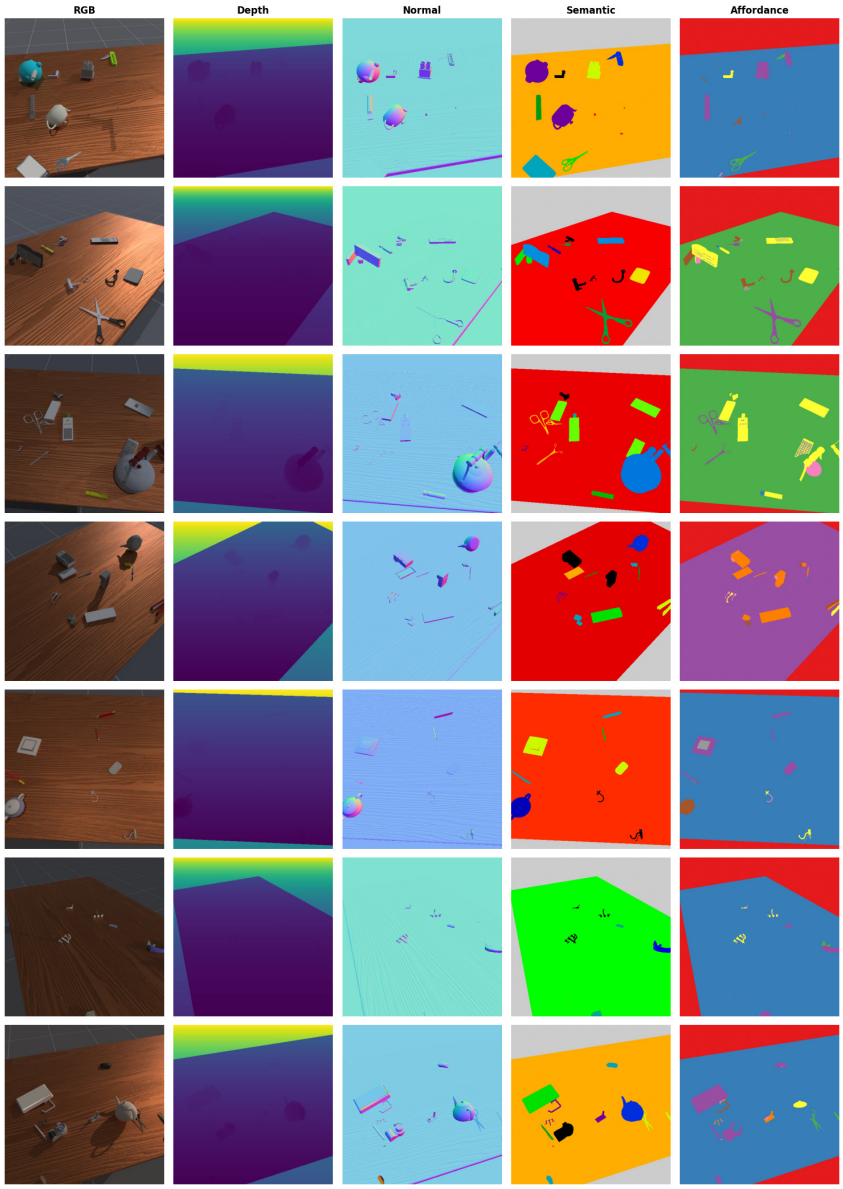


Figure 9: MOPS-Clutter dataset examples.

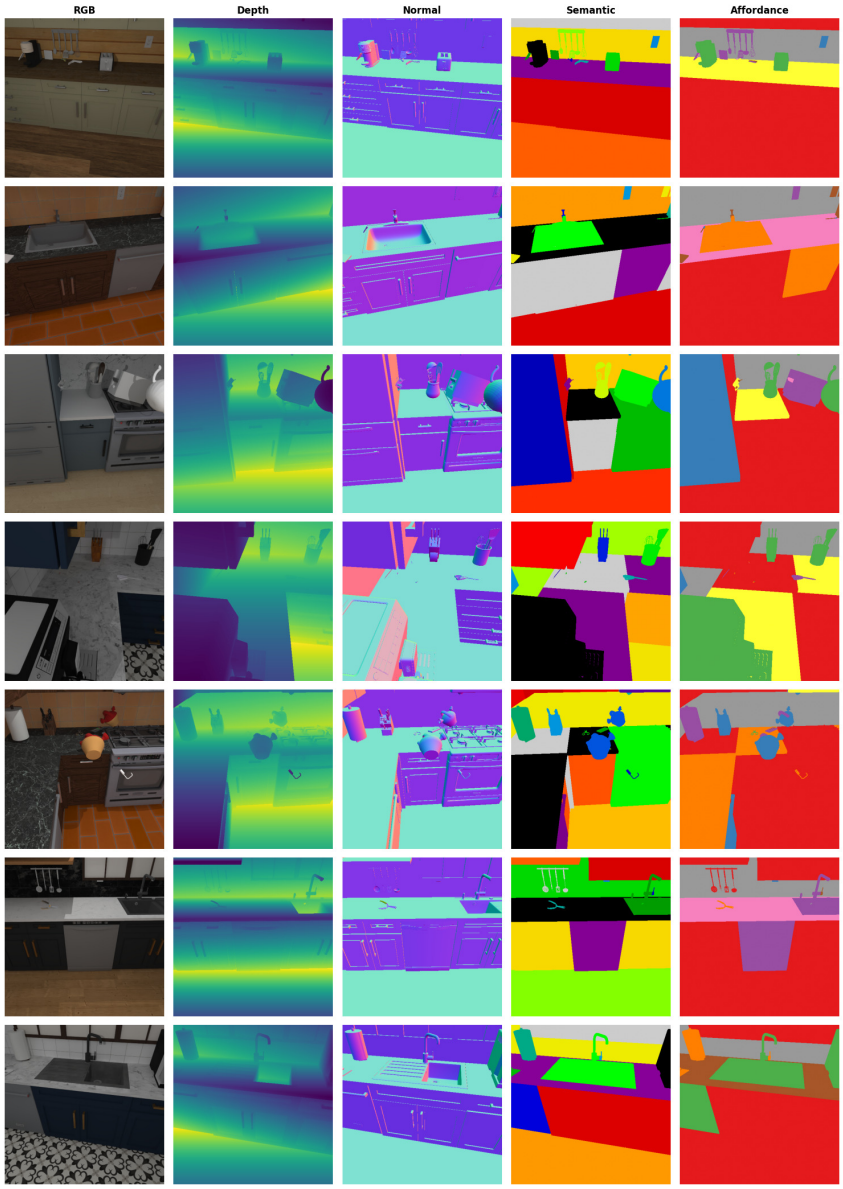


Figure 10: MOPS-Kitchen dataset examples.